#29/12/2025

# Create a MongoDB Cluster

Source code for practical case study here

A MongoDB cluster allows a MongoDB database to either horizontally scale across many servers with sharding, or to replicate data ensuring high availability with MongoDB replica sets, therefore enhancing the overall performance and reliability of the MongoDB cluster.

In this article we will show you how to create a MongoDB cluster locally for developing purposes with Docker.

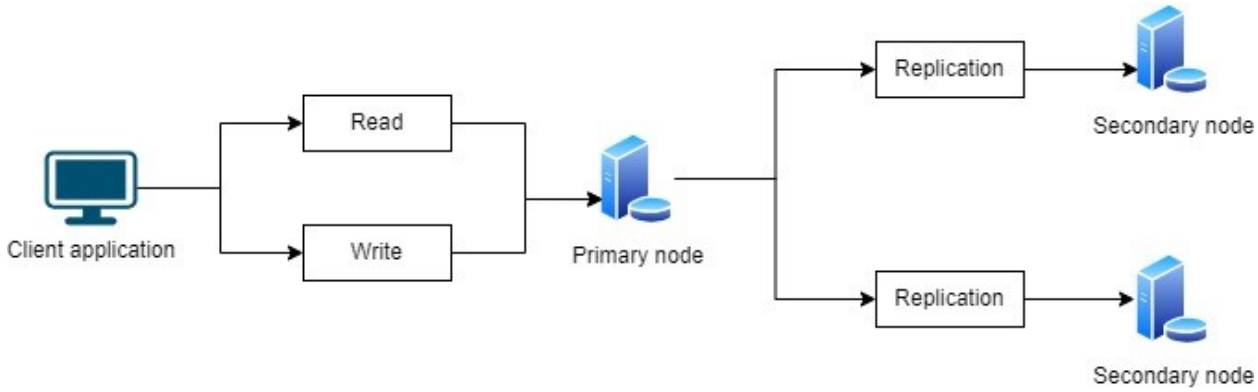
Author of the article: *Iker Elg, Alzaa*

https://github.com/kraftcoding

MongoDB is a general-purpose database that was built with the web in mind.

Amongst other things, it offers high availability when used in clusters, also called replica sets. A replica set is a group of MongoDB servers, called nodes, containing an identical copy of the data.

If one of the servers fails, the other two will pick up the load while the crashed one restarts.




There are many different ways to create a MongoDB cluster. In this article we will provide you with the necessary instructions to create your own MongoDB Docker cluster locally.

The easiest way is by using MongoDB Atlas, the database-as-a-service offering by MongoDB. With just a few clicks, you can create your free cluster, hosted in the cloud.

However, if you need to experiment with MongoDB clusters, you can use Docker to create a cluster on your personal computer. If you do not want to install MongoDB on your laptop, you can use Docker to run your cluster.

Docker is an application to launch containers, packages that contain applications along with all the required dependencies necessary to run them.

Here the steps to create MongoDB Cluster with initial data.

First we need to run Docker Desktop application, and after that, we pull the MongoDb official image from Docker Hub to our local host.

### Create a Docker network

Next step is to create a Docker network. This network will let each of your containers to run and see each other. To create a network, run the docker network create command.

```
docker network create mongoCluster
```

### Start MongoDB Instances

You are now ready to start your first container with MongoDB. To start the container, use the docker run command.

```
docker run -d --rm -p 27017:27017 --name
mongo1 --network mongoCluster mongo:latest
mongod --replSet myReplicaSet --bind_ip
localhost,mongo1
```

The command that will be executed once, starts the containers, and it will create a new mongod instance ready for a replica set.

Then, start two other containers, you will need to use a different name and a different port for those two.

```
docker run -d --rm -p 27018:27017 --name
mongo2 --network mongoCluster mongo:latest
```

```
mongod --replSet myReplicaSet --bind_ip
localhost,mongo2

docker run -d --rm -p 27019:27017 --name
mongo3 --network mongoCluster mongo:latest
mongod --replSet myReplicaSet --bind_ip
localhost,mongo3
```

You can use `docker ps` to validate that they are running or open the Docker application and check the containers.

### Initiate the Replica Set

The next step is to create the actual replica set with the three members.

To do so, you will need to use the MongoDB Shell. This CLI (command-line interface) tool is available with the default MongoDB installation or installed independently.

However, if you don't have the tool installed on your laptop, it is possible to use mongosh available inside containers with the docker exec command like this:

```
docker exec -it mongo1 mongosh --eval
"rs.initiate({
 _id: 'myReplicaSet',
 members: [
   {_id: 0, host: 'mongo1'},
   {_id: 1, host: 'mongo2'},
   {_id: 2, host: 'mongo3'}
 ]
})"
```

If you get an error because of the single quotes, try executing the command with double quotes instead or execute mongosh without the --eval parameter and then type the rs.initiate() command directly in the CLI.

This command tells Docker to run the mongosh tool inside the container named mongo1. Then `mongosh` will try to evaluate the `rs.initiate()` command to initiate the replica set.

If the command was successfully executed, you should see a message from the mongosh CLI indicating the version numbers of MongoDB and `mongosh`, followed by a message indicating: { ok: 1 }.

### Test and Verify the Replica Set

You should now have a running replica set.

If you want to verify that everything was configured correctly, you can use the mongosh CLI tool to evaluate the `rs.status()` instruction. This will provide you with the status of your replica set, including the list of members.

```
docker exec -it mongo1 mongosh --eval
"rs.status()"
```

If we launch `mongosh` comand, we will see that by default we will be connected to the primary node.

As we can notice, the connection string to test from visual studio is:

```
mongodb://127.0.0.1:27017/?
directConnection=true&serverSelectionTimeo
utMS=2000&appName=mongosh+2.5.10
```

And to test inside the container in Docker is:

```
is:"mongodb://host.docker.internal:27017/?
directConnection=true&serverSelectionTimeo
utMS=6000&appName=mongosh+2.5.10",
"Identity"
```